

Lab Assignment

C17 — STL `map<T>` — Core Operations

insert • emplace • operator[] • find • count • at • erase • clear • size • empty • iterate

1. Objective

This lab explores the core operations of the STL `map<string, int>` container. A map stores key-value pairs in sorted key order. Each key is unique — inserting a duplicate key has no effect. The map provides fast $O(\log n)$ lookup by key using a balanced binary tree internally.

All experiments use the Springfield Power Plant employees as data — character names as keys and their ages or salaries as values.

By the end of this lab you will be able to:

- ▶ Define and populate a map using an initializer list, `insert()`, `emplace()`, and `operator[]`.
- ▶ Access values safely using `at()` and unsafely using `operator[]`.
- ▶ Traverse a map using a range-for loop and an iterator.
- ▶ Search for keys using `find()` and `count()`.
- ▶ Remove entries using `erase()` by key and by iterator.
- ▶ Clear all entries using `clear()` and check emptiness with `empty()`.
- ▶ Understand the difference between `insert()`, `emplace()`, and `operator[]` when a key already exists.

2. Background

A map stores entries as **key-value pairs**. Every entry has two parts:

- ▶ **Key** — must be unique across the map. Determines sort order. Cannot be changed after insertion.
- ▶ **Value** — the data associated with the key. Can be read and modified freely.

When you iterate over a map the entries always appear in **ascending key order**, regardless of the order they were inserted. Each entry is a `pair<const Key, Value>`, accessed via `it->first` (key) and `it->second` (value).

3. Quick Reference – map<T> Methods

Method	Description
m[key] = val	Insert or UPDATE. If key exists, overwrites value. If not, inserts with val.
m.insert({key, val})	Insert ONLY. Ignored if key already exists. Returns pair<iterator, bool>.
m.emplace(key, val)	Same as insert but constructs the pair in place. Also ignored on duplicate.
m.at(key)	Returns value by reference. Throws out_of_range if key not found.
m.find(key)	Returns iterator to the entry, or end() if key not found.
m.count(key)	Returns 1 if key exists, 0 if not. (Always 0 or 1 for map.)
m.erase(key)	Removes the entry with that key. Returns number of entries removed (0 or 1).
m.erase(iterator)	Removes the entry pointed to by the iterator.
m.clear()	Removes all entries. size() becomes 0.
m.size()	Returns the number of key-value pairs currently stored.
m.empty()	Returns true if size() == 0.
m.begin() / m.end()	Iterators for traversal. Entries arrive in sorted key order.
m.rbegin() / m.rend()	Reverse iterators. Entries arrive in descending key order.

The critical difference between insert methods:

Method	Key exists?	Behavior
m[key] = val	Yes	OVERWRITES the existing value
m[key] = val	No	Inserts new entry with val
m.insert({key,val})	Yes	IGNORES the request — old value preserved
m.insert({key,val})	No	Inserts new entry with val
m.emplace(key,val)	Yes	IGNORES the request — old value preserved
m.emplace(key,val)	No	Inserts new entry with val

4. Required Experiments

Experiment 01 — Define and Populate

Four ways to add entries to a map. Each method has different behavior when the key already exists.

Tasks:

- ▶ Create `simpsonsMap` using an initializer list with four Springfield characters and their ages.
- ▶ Add a new entry using `operator[]`.
- ▶ Add a new entry using `insert({key, val})`.
- ▶ Add a new entry using `emplace(key, val)`.
- ▶ Print the map and its size after all insertions.
- ▶ Observe: the map prints in alphabetical order by name regardless of insertion order.

```
void experiment01() {
    cout << "=== Experiment 01: Define and Populate ===\n";

    // Method 1 -- initializer list
    map<string, int> simpsonsMap = {
        { "Homer Simpson", 45 },
        { "Marge Simpson", 42 },
        { "Bart Simpson", 10 },
        { "Lisa Simpson", 8 },
    };

    // Method 2 -- operator[]
    simpsonsMap["Maggie Simpson"] = 1;

    // Method 3 -- insert()
    simpsonsMap.insert({ "Ned Flanders", 60 });

    // Method 4 -- emplace()
    simpsonsMap.emplace("Montgomery Burns", 104);

    cout << "Map size: " << simpsonsMap.size() << "\n";
    cout << "Entries (sorted by name):\n";
    for (const auto& entry : simpsonsMap)
        cout << "    " << entry.first
            << " -> " << entry.second << "\n";
}
```

Experiment 02 — Access: operator[], at(), and find()

Three ways to read a value by key — each with different behavior when the key does not exist.

Tasks:

- ▶ Use `operator[]` to read Homer's age — note the danger if the key is missing.
- ▶ Use `at()` to read Marge's age — wraps in try-catch to catch `out_of_range`.
- ▶ Use `find()` to search for Bart — check the result against `end()` before using it.
- ▶ Use `find()` to search for a character not in the map — confirm it returns `end()`.
- ▶ Use `count()` to check whether a key exists without retrieving the value.

```
void experiment02() {
    cout << "=== Experiment 02: Access ===\n";

    map<string, int> simpsonsMap = {
        { "Homer Simpson", 45 }, { "Marge Simpson", 42 },
        { "Bart Simpson", 10 }, { "Lisa Simpson", 8 },
    };

    // operator[] -- DANGER: inserts empty entry if key is missing!
    cout << "Homer age (operator[]): " << simpsonsMap["Homer Simpson"] << "\n";

    // at() -- safe read, throws if key not found
    try {
        cout << "Marge age (at): " << simpsonsMap.at("Marge Simpson") <<
"\n";
        cout << "Burns age (at): " << simpsonsMap.at("Montgomery Burns") <<
"\n";
    }
    catch (const out_of_range& e) {
        cout << "    at() error: " << e.what() << "\n";
    }

    // find() -- safe lookup, returns end() if not found
    auto it = simpsonsMap.find("Bart Simpson");
    if (it != simpsonsMap.end())
        cout << "Found: " << it->first << " age=" << it->second << "\n";
    else
        cout << "Bart not found.\n";

    it = simpsonsMap.find("Krusty the Clown");
    if (it != simpsonsMap.end())
        cout << "Found Krusty\n";
    else
        cout << "Krusty not found (expected).\n";

    // count() -- existence check without retrieval
    cout << "Homer in map: " << (simpsonsMap.count("Homer Simpson") ? "YES" :
"NO") << "\n";
    cout << "Krusty in map: " << (simpsonsMap.count("Krusty the Clown") ? "YES" :
"NO") << "\n";
}
```

Experiment 03 — The Duplicate Key Trap

This experiment demonstrates the critical difference between `operator[]`, `insert()`, and `emplace()` when a key already exists.

Tasks:

- ▶ Insert Homer with age 45. Then try to insert Homer again with age 99 using `operator[]` — confirm the value IS overwritten.
- ▶ Insert Marge with age 42. Then try to insert Marge again with age 99 using `insert()` — confirm the value is NOT overwritten.
- ▶ Check the `bool` in the return value of `insert()` to see whether insertion succeeded.
- ▶ Repeat the duplicate test with `emplace()` — same behavior as `insert()`.

```
void experiment03() {
    cout << "=== Experiment 03: Duplicate Key Trap ===\n";

    map<string, int> m;

    // operator[] OVERWRITES on duplicate
    m["Homer Simpson"] = 45;
    cout << "Homer inserted with age 45: " << m["Homer Simpson"] << "\n";
    m["Homer Simpson"] = 99;           // overwrites!
    cout << "Homer after operator[]=99: " << m["Homer Simpson"] << "\n";

    // insert() IGNORES duplicate -- original value preserved
    m.insert({ "Marge Simpson", 42 });
    cout << "Marge inserted with age 42: " << m["Marge Simpson"] << "\n";

    auto result = m.insert({ "Marge Simpson", 99 }); // ignored!
    cout << "insert(Marge,99) succeeded: "
        << (result.second ? "YES" : "NO") << "\n";
    cout << "Marge age unchanged: " << m["Marge Simpson"] << "\n";

    // emplace() -- same behavior as insert() on duplicate
    m.emplace("Bart Simpson", 10);
    auto result2 = m.emplace("Bart Simpson", 99); // ignored!
    cout << "emplace(Bart,99) succeeded: "
        << (result2.second ? "YES" : "NO") << "\n";
    cout << "Bart age unchanged: " << m["Bart Simpson"] << "\n";
}
```

Experiment 04 — Traverse: Range-for and Iterator

A map always delivers entries in ascending key order. Both traversal styles demonstrate this.

Tasks:

- Traverse using a range-for loop with `const auto& entry` — print key and value.
- Traverse using a forward iterator — use `it->first` and `it->second`.
- Traverse in reverse using `rbegin()` and `rend()` — entries arrive in descending key order.
- During traversal, modify a value by dereferencing the iterator: `it->second = newValue`.

```
void experiment04() {
    cout << "=== Experiment 04: Traversal ===\n";

    map<string, int> simpsonsMap = {
        { "Homer Simpson",    45 },
        { "Marge Simpson",    42 },
        { "Bart Simpson",     10 },
        { "Lisa Simpson",     8 },
        { "Montgomery Burns", 104 },
    };

    // Style 1 -- range-for (most concise)
    cout << "Range-for (A to Z):\n";
    for (const auto& entry : simpsonsMap)
        cout << " " << entry.first << " -> " << entry.second << "\n";

    // Style 2 -- forward iterator
    cout << "\nForward iterator:\n";
    for (auto it = simpsonsMap.begin(); it != simpsonsMap.end(); ++it)
        cout << " " << it->first << " -> " << it->second << "\n";

    // Style 3 -- reverse iterator (Z to A)
    cout << "\nReverse iterator (Z to A):\n";
    for (auto it = simpsonsMap.rbegin(); it != simpsonsMap.rend(); ++it)
        cout << " " << it->first << " -> " << it->second << "\n";

    // Modify a value during iterator traversal
    // (key is const -- cannot be changed, but value can)
    for (auto it = simpsonsMap.begin(); it != simpsonsMap.end(); ++it)
        it->second += 1; // everyone gets a birthday!
    cout << "\nAfter birthday (+1 to all ages):\n";
    for (const auto& e : simpsonsMap)
        cout << " " << e.first << " -> " << e.second << "\n";
}
```

Experiment 05 — erase() and clear()

`erase()` removes one entry by key or by iterator. `clear()` removes all entries at once. Unlike vector, erasing from a map does not invalidate other iterators.

Tasks:

- ▶ Erase an entry by key using `erase("Bart Simpson")`. Confirm with `count()`.
- ▶ Erase an entry by iterator: find Homer then pass the iterator to `erase()`.
- ▶ Erase all entries whose age is below 18 using an iterator loop.
- ▶ Call `clear()` and verify `size()` is 0 and `empty()` is true.

```
void experiment05() {
    cout << "=== Experiment 05: erase() and clear() ===\n";

    map<string, int> simpsonsMap = {
        { "Homer Simpson",    45 },
        { "Marge Simpson",    42 },
        { "Bart Simpson",     10 },
        { "Lisa Simpson",     8 },
        { "Maggie Simpson",   1 },
        { "Montgomery Burns", 104 },
    };
    cout << "Initial size: " << simpsonsMap.size() << "\n";

    // Erase by key
    simpsonsMap.erase("Bart Simpson");
    cout << "After erase(Bart): size=" << simpsonsMap.size()
        << "   Bart still there: " << (simpsonsMap.count("Bart Simpson") ? "YES" :
"NO") << "\n";

    // Erase by iterator
    auto it = simpsonsMap.find("Homer Simpson");
    if (it != simpsonsMap.end())
        simpsonsMap.erase(it);
    cout << "After erase(Homer iterator): size=" << simpsonsMap.size() << "\n";

    // Erase all entries with age < 18
    for (auto it2 = simpsonsMap.begin(); it2 != simpsonsMap.end(); ) {
        if (it2->second < 18)
            it2 = simpsonsMap.erase(it2); // returns iterator to next entry
        else
            ++it2;
    }
    cout << "After erase(age<18):\n";
    for (const auto& e : simpsonsMap)
        cout << "   " << e.first << " -> " << e.second << "\n";

    // clear()
    simpsonsMap.clear();
    cout << "After clear(): size=" << simpsonsMap.size()
        << "   empty=" << (simpsonsMap.empty() ? "true" : "false") << "\n";
}
```

Experiment 06 — Practical Application: Salary Database

This experiment uses a `map<string, double>` to build a mini salary database for the Power Plant. It demonstrates insertion, lookup, update, and a summary query all in one realistic scenario.

Tasks:

- ▶ Populate the salary map with eight Power Plant employees.
- ▶ Look up Homer's salary using `find()`.
- ▶ Give Homer a raise using `operator[]` (update in place).
- ▶ Try to lower Burns's salary using `insert()` — confirm it fails (key exists).
- ▶ Count how many employees earn more than \$60,000.
- ▶ Find and print the highest-paid employee by scanning all entries.

```
void experiment06() {
    cout << "=== Experiment 06: Salary Database ===\n";

    map<string, double> salaryMap = {
        { "Montgomery Burns", 1000000.00 },
        { "Waylon Smithers",   95000.00 },
        { "Homer Simpson",     52000.00 },
        { "Lenny Leonard",     54000.00 },
        { "Carl Carlson",      68000.00 },
        { "Frank Grimes",      58000.00 },
        { "Professor Frink",   88000.00 },
        { "Hans Moleman",      22000.00 },
    };

    // Lookup
    auto it = salaryMap.find("Homer Simpson");
    if (it != salaryMap.end())
        cout << "Homer salary: $" << it->second << "\n";

    // Update -- give Homer a raise
    salaryMap["Homer Simpson"] += 5000.00;
    cout << "Homer after raise: $" << salaryMap["Homer Simpson"] << "\n";

    // Try to lower Burns with insert() -- will be ignored
    auto result = salaryMap.insert({ "Montgomery Burns", 1.00 });
    cout << "Burns insert(1.00) succeeded: "
        << (result.second ? "YES -- salary changed!" : "NO -- salary unchanged")
        << "\n";
    cout << "Burns salary: $" << salaryMap["Montgomery Burns"] << "\n";

    // Count employees earning more than $60,000
    int highEarnings = 0;
    for (const auto& e : salaryMap)
        if (e.second > 60000.00) highEarnings++;
    cout << "Employees earning > $60,000: " << highEarnings << "\n";

    // Find highest-paid employee
    auto best = salaryMap.begin();
    for (auto it2 = salaryMap.begin(); it2 != salaryMap.end(); ++it2)
        if (it2->second > best->second) best = it2;
    cout << "Highest paid: " << best->first
        << "  $" << best->second << "\n";
}
```

main() — Call All Experiments

```
#include <iostream>
#include <map>
#include <string>
using namespace std;

int main() {
    experiment01();
    experiment02();
    experiment03();
    experiment04();
    experiment05();
    experiment06();
    cout << "\nAll done!\n";
    return 0;
}
```

5. Expected Output (partial)

```
=== Experiment 01: Define and Populate ===
Map size: 7
Entries (sorted by name):
  Bart Simpson      -> 10
  Homer Simpson     -> 45
  Lisa Simpson      -> 8
  Maggie Simpson    -> 1
  Marge Simpson     -> 42
  Montgomery Burns  -> 104
  Ned Flanders      -> 60

=== Experiment 02: Access ===
Homer age (operator[]): 45
Marge age (at):         42
  at() error: key not found
Found: Bart Simpson age=10
Krusty not found (expected).
Homer in map:  YES
Krusty in map:  NO

=== Experiment 03: Duplicate Key Trap ===
Homer inserted with age 45: 45
Homer after operator[]=99: 99
Marge inserted with age 42: 42
insert(Marge,99) succeeded: NO
Marge age unchanged:       42
emplace(Bart,99) succeeded: NO
Bart age unchanged:        10

=== Experiment 04: Traversal ===
Range-for (A to Z):
  Bart Simpson      -> 10
  Homer Simpson     -> 45
  Lisa Simpson      -> 8
  Marge Simpson     -> 42
  Montgomery Burns  -> 104
...

=== Experiment 05: erase() and clear() ===
Initial size: 6
After erase(Bart): size=5  Bart still there: NO
After erase(Homer iterator): size=4
After erase(age<18):
  Marge Simpson     -> 42
  Montgomery Burns  -> 104
After clear(): size=0  empty=true

=== Experiment 06: Salary Database ===
Homer salary: $52000
Homer after raise: $57000
Burns insert(1.00) succeeded: NO -- salary unchanged
Burns salary: $1000000
Employees earning > $60,000: 4
Highest paid: Montgomery Burns  $1000000
```

6. Common Mistakes

Pitfalls to avoid

- ▶ **Using `operator[]` to read a missing key** — `m["unknown"]` silently inserts a zero-value entry and returns 0. Use `find()` or `at()` for safe reads.
- ▶ **Expecting `insert()` to update an existing entry** — `insert()` and `emplace()` are no-ops on duplicate keys. Use `operator[]` when you want to overwrite.
- ▶ **Modifying the key through an iterator** — keys are `const`. Only `it->second` (the value) can be changed during traversal.
- ▶ **Not checking `find()` result before dereferencing** — if `find()` returns `end()`, dereferencing it is undefined behavior. Always compare to `end()` first.
- ▶ **Assuming map order matches insertion order** — map always sorts by key. Use `unordered_map` if you need insertion order or faster average lookups.

7. Reflection Questions

Answer each question in 2–3 sentences:

- ▶ 1. What is the danger of using `operator[]` to read a value when you are not sure the key exists?
- ▶ 2. Explain the difference in behavior between `insert()` and `operator[]` when the key already exists.
- ▶ 3. Why are map keys always `const`? What problem would arise if you could change a key after insertion?
- ▶ 4. What does `find()` return when the key is not found, and why is it important to check this before using the result?
- ▶ 5. When would you choose `unordered_map` over `map`? What do you give up by making that choice?

8. Self-Assessment: Quality Control

Before *submitting* your work review each item below. A check on every line means your lab is complete.

☐	Item
☐	Experiment 01 — four insertion methods compile and print entries in sorted key order.
☐	Experiment 02 — <code>operator[]</code> , <code>at()</code> with exception caught, <code>find()</code> , and <code>count()</code> all demonstrated.
☐	Experiment 03 — duplicate key behavior confirmed for <code>operator[]</code> , <code>insert()</code> , and <code>emplace()</code> .
☐	Experiment 04 — three traversal styles (range-for, forward iterator, reverse iterator) produce correct sorted output.
☐	Experiment 04 — values successfully modified through iterator during traversal.
☐	Experiment 05 — erase by key, by iterator, and by condition all work correctly; <code>clear()</code> verified.
☐	Experiment 06 — all six salary tasks completed and output matches expected results.
☐	Code compiles with no errors and no warnings.
☐	<code>find()</code> return value is always checked against <code>end()</code> before dereferencing.
☐	<code>operator[]</code> is never used for read-only access when the key may be missing.
☐	Reflection questions answered in 2–3 sentences each.

Key insight

The map's most important design decision is that **`operator[]` always succeeds** — it never throws, never returns null, and never fails. The price is that it silently creates entries. Knowing when to use `operator[]`, `at()`, or `find()` is the single most important skill for working with maps correctly.